

Racing Game

Projektdokumentation – Architektur & Klassenstruktur

Q12 · Informatik 1inf1 · Hr. Stark / Hr. Leebmann

Verantwortlich: Jakob Grätz et al.

Sprache: Java · Perspektive: Top-Down

1. Zielsetzung & Produktübersicht

Entwicklung eines funktionsfähigen Autorennspiels mit Vogelperspektive in Java. Das Spiel soll als Präsentationsprojekt der Q12 sowie zum Zeitvertreib dienen und folgende Kernfunktionen abdecken:

- Beschleunigen, Bremsen, Lenken mit physikalisch plausibler Bewegung
- Prozedural generierte Rennstrecken mit Checkpoints und Rangliste
- Kollisionserkennung und -auflösung
- Zeit- und Rundensystem, verschiedene Fahrzeuge
- Gegnersystem: weitere Spieler und KI-gesteuerte Gegner
- Grafische Benutzeroberfläche mit HUD und Spielmenü
- Persistenz von Spielständen, Rekorden und Bestenlisten
- Audio: Musik, Fahrgeräusche, Effekte
- Optionale Erweiterungen: Boosts, Multiplayer-Backend

Qualitätsanforderungen

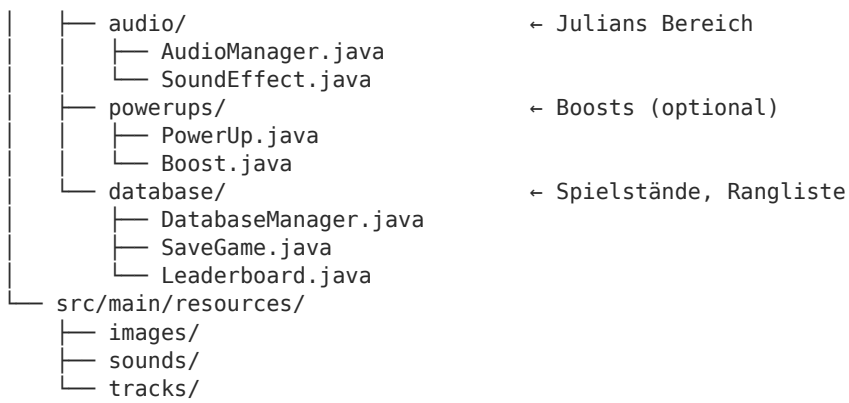
- **Zuverlässigkeit:** keine gravierenden Mängel im Endprodukt
- **Wartbarkeit:** klar strukturierter, lesbarer Code; konsequente Pakettrennung
- **Sicherheit:** keine Beeinträchtigung des Nutzergeräts
- **Technik:** Java, Verzicht auf obskure Drittparteibibliotheken

2. Projektstruktur

Das Projekt folgt dem Maven-Standardlayout. Die fachliche Trennung in Pakete spiegelt die einzelnen Verantwortungsbereiche im Team wider und minimiert Merge-Konflikte:

```

racinggame/
├── src/main/java/com/racinggame/
│   ├── Main.java                ← Einstiegspunkt
│   ├── core/                    ← Spielsteuerung
│   │   ├── Game.java
│   │   ├── GameLoop.java
│   │   └── GameState.java
│   ├── entities/                ← Fahrzeuge & Spieler (abstract)
│   │   ├── Vehicle.java
│   │   ├── PlayerVehicle.java
│   │   ├── AIVehicle.java
│   │   └── VehicleType.java      (enum)
│   ├── physics/                 ← Bewegung & Kollisionen
│   │   ├── PhysicsEngine.java
│   │   ├── Collision.java
│   │   └── Vector2D.java
│   ├── track/                   ← Strecke
│   │   ├── Track.java
│   │   ├── TrackGenerator.java  (prozedural)
│   │   ├── TrackSegment.java
│   │   └── Checkpoint.java
│   ├── ui/                      ← Grafik & Menüs
│   │   ├── GameWindow.java
│   │   ├── Renderer.java
│   │   ├── MainMenu.java
│   │   └── HUD.java
│   └── input/
│       └── InputHandler.java
```



3. Klassendiagramm (Übersicht)

Das folgende UML-Klassendiagramm zeigt sämtliche Klassen mit Attributen, Methoden und Beziehungen. Symbolik: + public, – private, # protected. Geschlossenes Dreieck = Vererbung, gefüllter Rhombus = Komposition, leerer Rhombus = Aggregation, gestrichelte Linie = «creates».

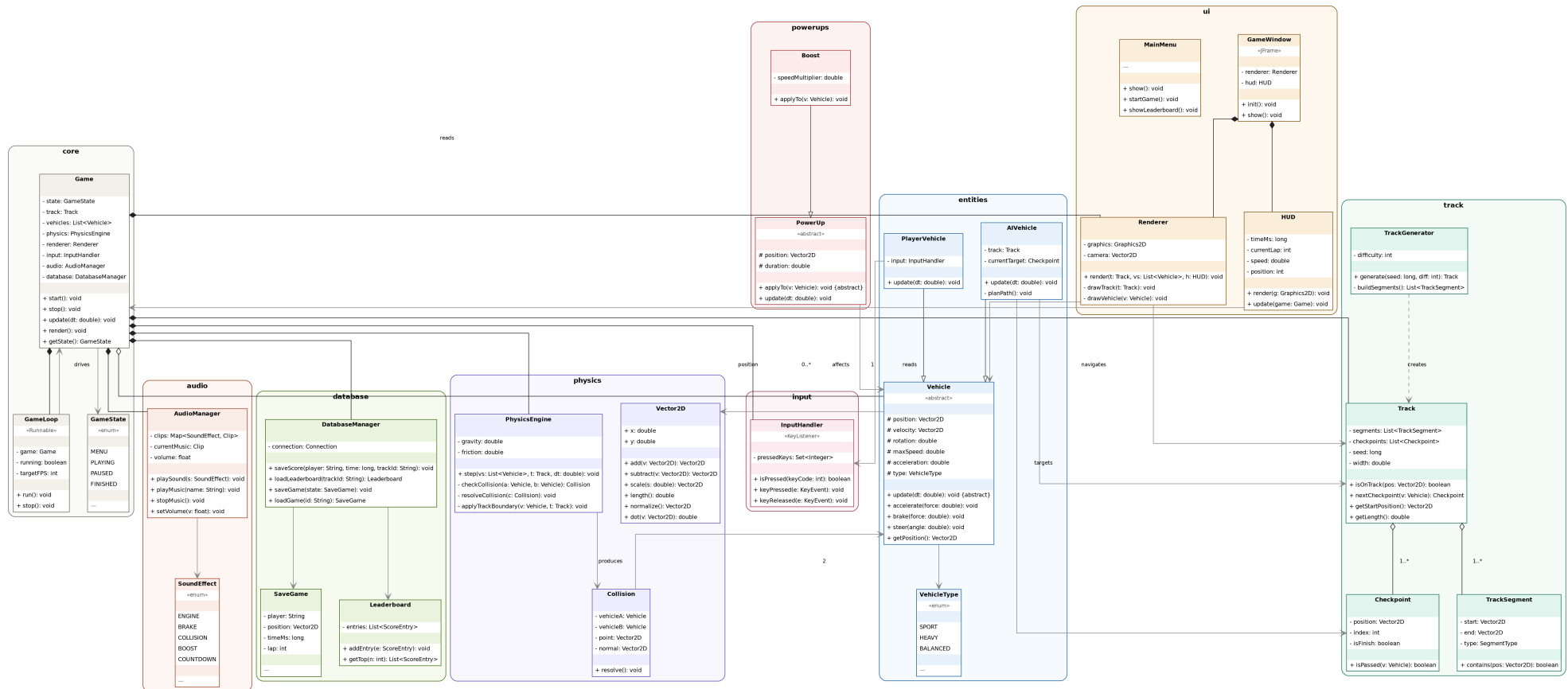


Abb. 1 · Vollständiges UML-Klassendiagramm der Racing-Game-Architektur

4. Klassen im Detail

Pro Paket werden im Folgenden die Verantwortlichkeiten beschrieben und die Schnittstellen als Java-Skelett angegeben. Implementierungen sind bewusst leer – die Signaturen sind verbindlich, die Innereien werden im Team ausgearbeitet.

4.1 Paket core

Zentrale Spielsteuerung. *Game* orchestriert alle Subsysteme; *GameLoop* sorgt für eine konstante Aktualisierungsrate (Fixed Timestep, ca. 60 Hz); *GameState* beschreibt den aktuellen Lebenszyklus-Zustand.

```
public class Game {
    private GameState state;
    private Track track;
    private List<Vehicle> vehicles;
    private PhysicsEngine physics;
    private Renderer renderer;
    private InputHandler input;
    private AudioManager audio;
    private DatabaseManager database;

    public void start()           { /* GameLoop starten */ }
    public void stop()            { }
    public void update(double dt) { /* Logik aktualisieren */ }
    public void render()          { /* Frame zeichnen */ }
    public GameState getState()   { return state; }
}

public class GameLoop implements Runnable {
    private final Game game;
    private boolean running;
    private int targetFPS = 60;

    public void run() { /* Fixed-Timestep-Schleife */ }
    public void stop() { running = false; }
}

public enum GameState { MENU, PLAYING, PAUSED, FINISHED }
```

4.2 Paket entities

Alle beweglichen Spielobjekte. *Vehicle* ist abstrakte Basisklasse mit Position, Geschwindigkeit, Rotation und gemeinsamen Methoden für Beschleunigen/Bremsen/Lenken. *PlayerVehicle* liest Eingaben aus dem *InputHandler*, *AIVehicle* entscheidet anhand der nächsten Checkpoints autonom.

```
public abstract class Vehicle {
    protected Vector2D position;
    protected Vector2D velocity;
    protected double rotation;
    protected double maxSpeed;
    protected double acceleration;
    protected VehicleType type;

    public abstract void update(double dt);
    public void accelerate(double force) { }
    public void brake(double force)     { }
    public void steer(double angle)      { }
    public Vector2D getPosition()        { return position; }
}

public class PlayerVehicle extends Vehicle {
```

```

    private InputHandler input;
    @Override public void update(double dt) { /* Eingaben + Bewegung */ }
}

public class AIVehicle extends Vehicle {
    private Track track;
    private Checkpoint currentTarget;
    @Override public void update(double dt) { /* KI-Verhalten */ }
    private void planPath() { }
}

public enum VehicleType { SPORT, HEAVY, BALANCED }

```

4.3 Paket track

Streckenrepräsentation und prozedurale Generierung. Eine Strecke besteht aus einer geordneten Folge von *TrackSegments* und *Checkpoints*. Der *TrackGenerator* erzeugt aus einem Seed reproduzierbare, deterministische Strecken – das vereinfacht Debugging und Bestenlisten-Vergleichbarkeit.

```

public class Track {
    private List<TrackSegment> segments;
    private List<Checkpoint> checkpoints;
    private long seed;
    private double width;

    public boolean isOnTrack(Vector2D pos) { return false; }
    public Checkpoint nextCheckpoint(Vehicle v) { return null; }
    public Vector2D getStartPosition() { return null; }
    public double getLength() { return 0; }
}

public class TrackGenerator {
    private int difficulty;
    public Track generate(long seed, int diff) { return null; }
    private List<TrackSegment> buildSegments() { return null; }
}

public class TrackSegment {
    private Vector2D start, end;
    private SegmentType type;
    public boolean contains(Vector2D pos) { return false; }
}

public class Checkpoint {
    private Vector2D position;
    private int index;
    private boolean isFinish;
    public boolean isPassed(Vehicle v) { return false; }
}

```

4.4 Paket physics

Bewegungs- und Kollisionsberechnungen. *PhysicsEngine.step()* wird pro Frame vom GameLoop aufgerufen, integriert Geschwindigkeiten, prüft Streckengrenzen und löst Fahrzeug-zu-Fahrzeug-Kollisionen auf. *Vector2D* ist eine kleine, immutable-freundliche Vektorklasse für alle Positions- und Geschwindigkeitsoperationen.

```

public class PhysicsEngine {
    private double gravity;
    private double friction;

    public void step(List<Vehicle> vs, Track t, double dt) { }
    private Collision checkCollision(Vehicle a, Vehicle b) { return null; }
    private void resolveCollision(Collision c) { }
}

```

```

        private void applyTrackBoundary(Vehicle v, Track t)    { }
    }

    public class Vector2D {
        public double x, y;
        public Vector2D add(Vector2D v)           { return null; }
        public Vector2D subtract(Vector2D v)       { return null; }
        public Vector2D scale(double s)           { return null; }
        public double length()                     { return 0; }
        public Vector2D normalize()                { return null; }
        public double dot(Vector2D v)              { return 0; }
    }

    public class Collision {
        private Vehicle vehicleA, vehicleB;
        private Vector2D point, normal;
        public void resolve() { }
    }

```

4.5 Paket ui

Darstellung & Menüführung. *GameWindow* hostet das Swing-Fenster, *Renderer* zeichnet Strecke und Fahrzeuge in Vogelperspektive, *HUD* ist das Heads-Up-Display (Zeit, Runde, Geschwindigkeit, Position). Der *MainMenu*-Bereich ist von der Spielszene entkoppelt.

```

    public class GameWindow extends JFrame {
        private Renderer renderer;
        private HUD hud;
        public void init() { }
        public void show() { }
    }

    public class Renderer {
        private Graphics2D graphics;
        private Vector2D camera;
        public void render(Track t, List<Vehicle> vs, HUD h) { }
        private void drawTrack(Track t)                    { }
        private void drawVehicle(Vehicle v)                 { }
    }

    public class HUD {
        private long timeMs;
        private int currentLap;
        private double speed;
        private int position;
        public void render(Graphics2D g) { }
        public void update(Game game)   { }
    }

    public class MainMenu {
        public void show()           { }
        public void startGame()      { }
        public void showLeaderboard() { }
    }

```

4.6 Pakete audio & input

AudioManager kapselt Musik- und Effekt-Wiedergabe (Julians Verantwortungsbereich). *SoundEffect* ist ein Enum für sauber referenzierbare Klang-IDs. *InputHandler* implementiert *KeyListener* und stellt einen einfachen *isPressed(keyCode)*-Mechanismus bereit, den der *PlayerVehicle* pro Frame abfragt.

```

    public class AudioManager {
        private Map<SoundEffect, Clip> clips;
        private Clip currentMusic;
    }

```

```

    private float volume;

    public void playSound(SoundEffect s) { }
    public void playMusic(String name) { }
    public void stopMusic() { }
    public void setVolume(float v) { }
}

public enum SoundEffect { ENGINE, BRAKE, COLLISION, BOOST, COUNTDOWN }

public class InputHandler implements KeyListener {
    private Set<Integer> pressedKeys;
    public boolean isPressed(int keyCode) { return false; }
    @Override public void keyPressed(KeyEvent e) { }
    @Override public void keyReleased(KeyEvent e) { }
    @Override public void keyTyped(KeyEvent e) { }
}

```

4.7 Pakete database & powerups

Persistenz und optionale Power-Ups. Für die Datenbank reicht zunächst eine eingebettete Lösung (z.B. SQLite via JDBC oder einfach JSON-Dateien); die Schnittstelle bleibt davon unabhängig. Power-Ups sind eine optionale Erweiterung – die Basisklasse ist bewusst schlank gehalten.

```

public class DatabaseManager {
    private Connection connection;
    public void saveScore(String player, long timeMs, String trackId) { }
    public Leaderboard loadLeaderboard(String trackId) { return null; }
    public void saveGame(SaveGame state) { }
    public SaveGame loadGame(String id) { return null; }
}

public class SaveGame {
    private String player;
    private Vector2D position;
    private long timeMs;
    private int lap;
}

public class Leaderboard {
    private List<ScoreEntry> entries;
    public void addEntry(ScoreEntry e) { }
    public List<ScoreEntry> getTop(int n) { return null; }
}

public abstract class PowerUp {
    protected Vector2D position;
    protected double duration;
    public abstract void applyTo(Vehicle v);
    public void update(double dt) { }
}

public class Boost extends PowerUp {
    private double speedMultiplier;
    @Override public void applyTo(Vehicle v) { }
}

```


5. Rollenverteilung im Team

Die Aufteilung folgt sowohl dem Lastenheft als auch der Paketstruktur, sodass jedes Teammitglied einen klar abgegrenzten Verantwortungsbereich hat:

Bereich	Paket(e)	Verantwortliche
Audio & Klanggestaltung	audio	Julian
Design & Grafik (Sprites, UI-Look)	ui, resources/images	Jakob, Andreas, Dominik, Ilias
Spielkern & Physik	core, physics	Jakob G., Christoph
Strecke & Generator	track	Emanuel, Maxi
Fahrzeuge & KI	entities	Andreas, Christoph
Eingaben & HUD	input, ui (HUD)	frei zuteilbar
Datenbank & Rangliste	database	frei zuteilbar (spät einsteigend)

Gegenseitige Informationspflicht

Wie im Lastenheft festgehalten, ist jeder im Team verpflichtet, andere über relevante Änderungen zu informieren – insbesondere an Schnittstellen zwischen den Paketen *physics* ↔ *entities* ↔ *track*, da dort die meisten Merge-Konflikte entstehen werden.

6. Empfohlene Reihenfolge der Umsetzung

Damit nicht alle gleichzeitig auf unfertige Komponenten warten, lässt sich das Projekt in vier Iterationen agil umsetzen. Jede Iteration endet mit einem Stand, der vorgeführt werden kann:

Iteration 1 – Skelett

Klassengerüst aus diesem Dokument anlegen, Maven-Projekt aufsetzen, leere Methoden mit TODO-Kommentaren befüllen. Ziel: das Projekt kompiliert.

Iteration 2 – Bewegung in der Welt

GameLoop, *InputHandler*, *PlayerVehicle*, primitive *PhysicsEngine* (nur Bewegung, noch keine Kollisionen) und ein minimaler *Renderer*, der ein Auto auf leerer Fläche fahren lässt.

Iteration 3 – Strecke & Spielablauf

TrackGenerator, *Checkpoint*-System, Rundenzähler im *HUD*, Kollisionen mit Streckenrand und zwischen Fahrzeugen, erste *AIVehicle*-Implementierung.

Iteration 4 – Politur

AudioManager und Effekte, *DatabaseManager* für Bestenlisten, *MainMenu*, Sprite-Polishing, optional *Boost*-*Power-Ups*. Multiplayer-Backend nur, wenn Zeit übrig ist.

Nicht zu früh angehen

Multiplayer mit Backend ist im Lastenheft mit Fragezeichen markiert – diesen Punkt erst angehen, wenn der Singleplayer rund läuft. Andernfalls verschlingt Netzwerk-Synchronisation Zeit, die für die Pflichtfeatures fehlt.